

贺鑫帅第四周总结（4.7）

2025 年 4 月 7 日

1 实践操作 paddlepaddle

1. 数据集定义与加载 2. 模型组网 3. 模型训练与评估 4. 模型推理

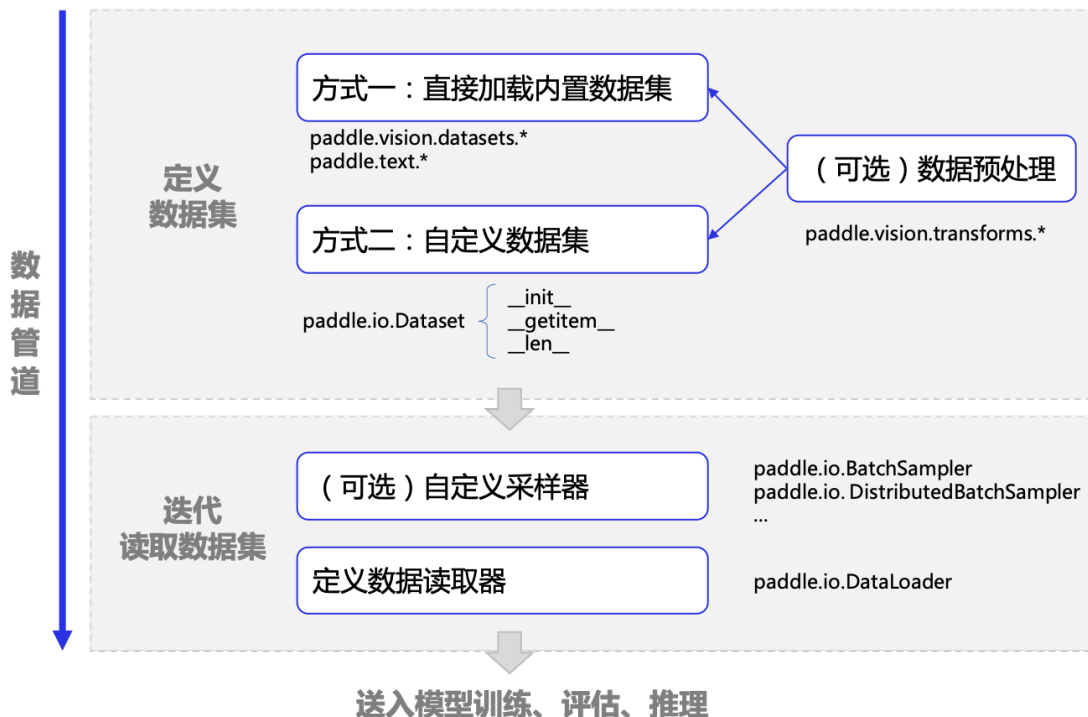
1.1 环境配置

```
[ ]: conda install paddlepaddle-gpu==3.0.0 paddlepaddle-cuda=12.6 -c paddle -c nvidia
```

```
[ ]: conda install matplotlib numpy
```

```
[ ]: import paddle  
paddle.utils.run_check()
```

1.2 数据集定义和加载



1.2.1 定义数据集

1. 内置 paddle.vision.datasets

```
[16]: print("cv dataset:\n",paddle.vision.datasets.__all__)
```

cv dataset:

```
['DatasetFolder', 'ImageFolder', 'MNIST', 'FashionMNIST', 'Flowers', 'Cifar10',  
'Cifar100', 'VOC2012']
```

```
[10]: from paddle.vision.transforms import Normalize  
transform=Normalize(mean=[127.5],std=[127.5],data_format="CHW")  
train_dataset=paddle.vision.datasets.MNIST(mode="train",transform=transform)  
test_dataset=paddle.vision.datasets.MNIST(mode="test",transform=transform)  
print("train_dataset_len:",len(train_dataset),"test_dataset_len:"  
↪ ,len(test_dataset))
```

```
train_dataset_len: 60000 test_dataset_len: 10000
```

2. 自定义数据集 可以使用 `paddle.io.Dataset`, 构建一个子类继承 `paddle.io.Dataset` 实现以下三个函数

1. `__init__` > 完成数据集初始化, 将磁盘岩本文件路径和对应标签映射到一个列表中
2. `__getitem__` > 定义指定 `index` 时如何获取样本数据, 最终返回单条数据
3. `len` > 返回数据集样本总数

1.2.2 迭代读取数据集

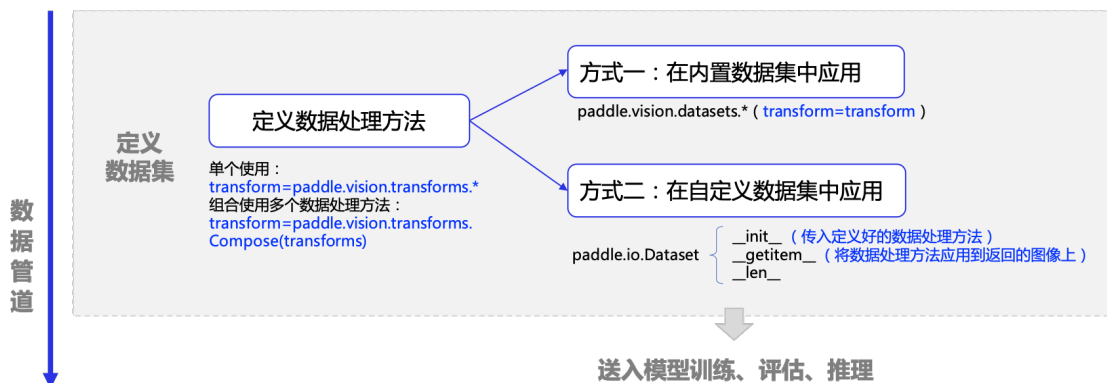
在飞桨框架中, 推荐使用 `paddle.io.DataLoader` API 对数据集进行多进程的读取, 并且可自动完成划分 `batch` 的工作。> 常用字段 > >- `dataset` >- `batch_size` >- `shuffle` >- `drop_last` >- `num_workers`
> 如果使用高层 API 的 `paddle.Model.fit` 读取数据集进行训练, 则只需定义数据集 `Dataset` 即可, 不需要再单独定义 `DataLoader`, 因为 `paddle.Model.fit` 中实际已经封装了一部分 `DataLoader` 的功能

[40]: # 定义初始化数据读取器

```
train_loader=paddle.io.DataLoader(  
    train_dataset,  
    batch_size=64,  
    shuffle=True,  
    num_workers=3,  
    drop_last=True,)  
for batch_id, data in enumerate(train_loader()):  
    images,labels=data  
    print(  
        "batch_id: {}, 训练数据 shape: {}, 标签数据 shape: {}".format(  
            batch_id, images.shape, labels.shape  
        )  
    )  
    break
```

batch_id: 0, 训练数据 shape: [64, 1, 28, 28], 标签数据 shape: [64, 1]

1.3 数据预处理



1.3.1 paddle.vision.transforms

```
[1]: import paddle

print("图像数据处理方法: ", paddle.vision.transforms.__all__)
```

图像数据处理方法: ['BaseTransform', 'Compose', 'Resize', 'RandomResizedCrop', 'CenterCrop', 'RandomHorizontalFlip', 'RandomVerticalFlip', 'Transpose', 'Normalize', 'BrightnessTransform', 'SaturationTransform', 'ContrastTransform', 'HueTransform', 'ColorJitter', 'RandomCrop', 'Pad', 'RandomAffine', 'RandomRotation', 'RandomPerspective', 'Grayscale', 'ToTensor', 'RandomErasing', 'to_tensor', 'hflip', 'vflip', 'resize', 'pad', 'affine', 'rotate', 'perspective', 'to_grayscale', 'crop', 'center_crop', 'adjust_brightness', 'adjust_contrast', 'adjust_hue', 'normalize', 'erase']

使用

定义

1. 单个使用

```
[4]: from paddle.vision.transforms import Resize

# 定义一个待使用的数据处理方法，这里定义了一个调整图像大小的方法
transform = Resize(size=28)
```

2. 组合使用

```
[5]: from paddle.vision.transforms import Compose, RandomRotation

# 定义待使用的数据处理方法，这里包括随机旋转、改变图片大小两个组合处理
transform = Compose([RandomRotation(10), Resize(size=32)])
```

应用

1. 内置数据集中

```
[7]: # 通过 transform 字段传递定义好的数据处理方法，即可完成对框架内置数据集的增强
train_dataset = paddle.vision.datasets.MNIST(mode="train", transform=transform)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[7], line 2
      1 # 通过 transform 字段传递定义好的数据处理方法，即可完成对框架内置数据集的增强
----> 2 train_dataset = paddle.vision.datasets.MNIST(mode="train",
      ↪ transform=transform)

NameError: name 'transform' is not defined
```

2. 自定义数据集中

对于自定义的数据集，可以在数据集中将定义好的数据处理方法传入 `init` 函数，将其定义为自定义数据集类的一个属性，然后在 `getitem` 中将其应用到图像上，如下述代码所示：

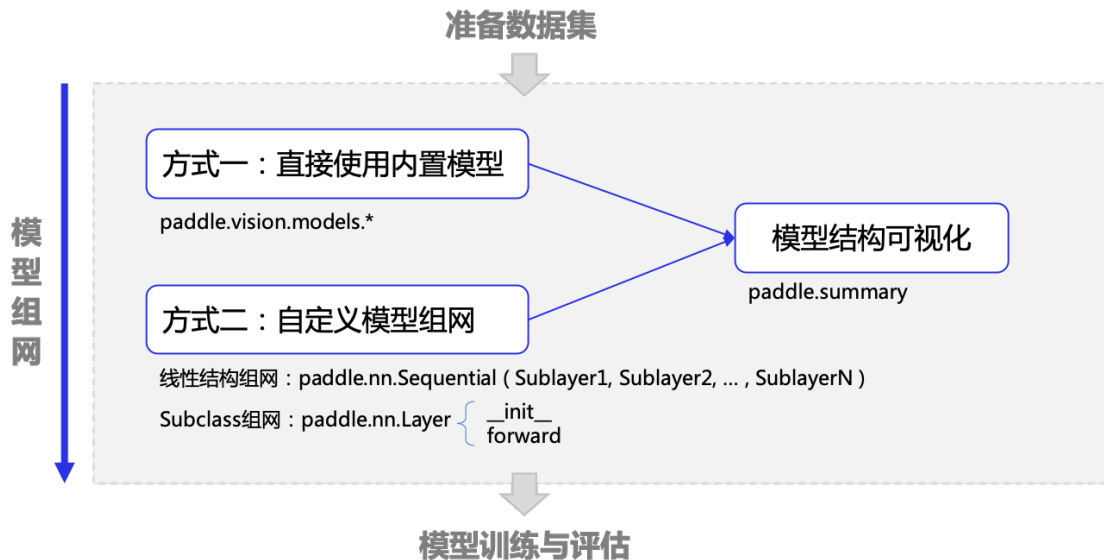
```
[9]: # def __init__(self, data_dir, label_path, transform=None):

#         # 1. 传入定义好的数据处理方法，作为自定义数据集类的一个属性
#         self.transform = transform

#     def __getitem__(self, index):

#         # 2. 应用数据处理方法到图像上
#         if self.transform is not None:
#             image = self.transform(image)
```

1.4 模型组网



1. 内置模型

```
[3]: # paddle.vision.models 内置的模型
import paddle
print("内置 model: ",paddle.vision.models.__all__)
```

```
内置 model: ['ResNet', 'resnet18', 'resnet34', 'resnet50', 'resnet101',
'resnet152', 'resnext50_32x4d', 'resnext50_64x4d', 'resnext101_32x4d',
'resnext101_64x4d', 'resnext152_32x4d', 'resnext152_64x4d', 'wide_resnet50_2',
'wide_resnet101_2', 'VGG', 'vgg11', 'vgg13', 'vgg16', 'vgg19', 'MobileNetV1',
'mobilenet_v1', 'MobileNetV2', 'mobilenet_v2', 'MobileNetV3Small',
'MobileNetV3Large', 'mobilenet_v3_small', 'mobilenet_v3_large', 'LeNet',
'DenseNet', 'densenet121', 'densenet161', 'densenet169', 'densenet201',
'densenet264', 'AlexNet', 'alexnet', 'InceptionV3', 'inception_v3',
'SqueezeNet', 'squeezenet1_0', 'squeezenet1_1', 'GoogLeNet', 'googlenet',
'ShuffleNetV2', 'shufflenet_v2_x0_25', 'shufflenet_v2_x0_33',
'shufflenet_v2_x0_5', 'shufflenet_v2_x1_0', 'shufflenet_v2_x1_5',
'shufflenet_v2_x2_0', 'shufflenet_v2_swish']
```

```
[24]: # 示例
lenet =paddle.vision.models.LeNet(num_classes=10)
paddle.summary(lenet,(1,1,28,28))
```

D:\anaconda\envs\paddle_env\Lib\site-packages\paddle\hapi\model_summary.py:336:

UserWarning: Your model was created in static graph mode, this may not get correct summary information!

```
input_size.append(tuple(input[key].shape))
```

Layer (type)	Input Shape	Output Shape	Param #
Conv2D-1	[[1, 1, 28, 28]]	[1, 6, 28, 28]	60
ReLU-1	[[1, 6, 28, 28]]	[1, 6, 28, 28]	0
MaxPool2D-1	[[1, 6, 28, 28]]	[1, 6, 14, 14]	0
Conv2D-2	[[1, 6, 14, 14]]	[1, 16, 10, 10]	2,416
ReLU-2	[[1, 16, 10, 10]]	[1, 16, 10, 10]	0
MaxPool2D-2	[[1, 16, 10, 10]]	[1, 16, 5, 5]	0
Linear-1	[[1, 400]]	[1, 120]	48,120
Linear-2	[[1, 120]]	[1, 84]	10,164
Linear-3	[[1, 84]]	[1, 10]	850
Total params: 61,610			
Trainable params: 61,610			
Non-trainable params: 0			
Input size (MB): 0.00			
Forward/backward pass size (MB): 0.11			
Params size (MB): 0.24			
Estimated Total Size (MB): 0.35			

```
[24]: {'total_params': np.int64(61610), 'trainable_params': np.int64(61610)}
```

如上报错了，大体意思是 cuda 和 cudnn 的问题，但是 cuda 是指定版本的，所以不可能有问题，最终确定是 cudnn 的问题，一直报错 cudnn64_9.dll 找不到，于是推测必须使用 cudnn9.x 重新安装。

但是还是报错，后来重启了 notebook 成功解决。

```
[4]: # 示例
lenet = paddle.vision.models.LeNet(num_classes=10)
paddle.summary(lenet, (1, 1, 28, 28))
```

Layer (type)	Input Shape	Output Shape	Param #
Conv2D-1	[[1, 1, 28, 28]]	[1, 6, 28, 28]	60
ReLU-1	[[1, 6, 28, 28]]	[1, 6, 28, 28]	0
MaxPool2D-1	[[1, 6, 28, 28]]	[1, 6, 14, 14]	0
Conv2D-2	[[1, 6, 14, 14]]	[1, 16, 10, 10]	2,416
ReLU-2	[[1, 16, 10, 10]]	[1, 16, 10, 10]	0
MaxPool2D-2	[[1, 16, 10, 10]]	[1, 16, 5, 5]	0
Linear-1	[[1, 400]]	[1, 120]	48,120
Linear-2	[[1, 120]]	[1, 84]	10,164
Linear-3	[[1, 84]]	[1, 10]	850
Total params: 61,610			
Trainable params: 61,610			
Non-trainable params: 0			
Input size (MB): 0.00			
Forward/backward pass size (MB): 0.11			
Params size (MB): 0.24			
Estimated Total Size (MB): 0.35			

```
[4]: {'total_params': np.int64(61610), 'trainable_params': np.int64(61610)}
```

基于 paddle.nn 自定义模型 经典模型可以满足一些简单深度学习任务的需求，然后更多情况下，需要使用深度学习框架构建一个自己的神经网络，这时可以使用飞桨框架 paddle.nn 下的 API 构建网络，该目录下定义了丰富的神经网络层和相关函数 API，如卷积网络相关的 Conv1D、Conv2D、Conv3D，循环神经网络相关的 RNN、LSTM、GRU 等，方便组网调用飞桨提供继承类（class）的方式构建网络，并提供了几个基类，如：paddle.nn.Sequential、paddle.nn.Layer 等，构建一个继承基类的子类，并在子类中添加层（layer，如卷积层、全连接层等）可实现网络的构建，不同基类对应不同的组网方式

1. 用 paddle.nn.Layer 组网

构建一些比较复杂的网络结构时，可以选择该方式，组网包括三个步骤：>1. 创建一个

继承自 `paddle.nn.Layer` 的类；>2. 在类的构造函数 `init` 中定义组网用到的神经网络层 (layer)；>3. 在类的前向计算函数 `forward` 中使用定义好的 layer 执行前向计算。

```
[26]: from paddle import nn
# 以 lenet 为例
class LeNet(nn.Layer):
    def __init__(self, num_classes=10):
        super().__init__()
        self.num_classes = num_classes
        self.features = nn.Sequential(
            nn.Conv2D(1, 6, 3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2D(2, 2),
            nn.Conv2D(6, 16, 5, stride=1, padding=0),
            nn.ReLU(),
            nn.MaxPool2D(2, 2),
        )
        if num_classes > 0:
            self.linear = nn.Sequential(
                nn.Linear(400, 120),
                nn.Linear(120, 84),
                nn.Linear(84, num_classes),
            )
        def forward(self, inputs):
            x = self.features(inputs)
            if self.num_classes > 0:
                x = paddle.flatten(x, 1)
                x = self.linear(x)
            return x
lenet = LeNet()
paddle.summary(lenet, (1, 1, 28, 28))
```

Layer (type)	Input Shape	Output Shape	Param #
Conv2D-13	[[1, 1, 28, 28]]	[1, 6, 28, 28]	60
ReLU-13	[[1, 6, 28, 28]]	[1, 6, 28, 28]	0

MaxPool2D-13	[[1, 6, 28, 28]]	[1, 6, 14, 14]	0
Conv2D-14	[[1, 6, 14, 14]]	[1, 16, 10, 10]	2,416
ReLU-14	[[1, 16, 10, 10]]	[1, 16, 10, 10]	0
MaxPool2D-14	[[1, 16, 10, 10]]	[1, 16, 5, 5]	0
Linear-13	[[1, 400]]	[1, 120]	48,120
Linear-14	[[1, 120]]	[1, 84]	10,164
Linear-15	[[1, 84]]	[1, 10]	850

Total params: 61,610
Trainable params: 61,610
Non-trainable params: 0

Input size (MB): 0.00
Forward/backward pass size (MB): 0.11
Params size (MB): 0.24
Estimated Total Size (MB): 0.35

```
[26]: {'total_params': np.int64(61610), 'trainable_params': np.int64(61610)}
```

在飞桨框架中内置了丰富的神经网络层，用类（class）的方式表示，构建模型时可直接作为实例添加到子类中，只需设置一些必要的参数，并定义前向计算函数即可，反向传播和参数保存由框架自动完成。

一定要注意大小写，如 Sequential, Layer

2. 使用 paddle.nn.Sequential 组网 构建顺序的线性网络结构时，可以选择该方式，只需要按模型的结构顺序，一层一层加到 paddle.nn.Sequential 子类中即可 > 类 layer 的 Sequential 部分

```
[9]: mnist = paddle.nn.Sequential(
    paddle.nn.Flatten(1,-1),
    # paddle.nn.Linear(784,512),
    paddle.nn.ReLU(),
    paddle.nn.Dropout(0.2),
    paddle.nn.Linear(784,10),
)
```

1.5 模型训练

训练包括多轮迭代（epoch），每轮迭代遍历一次训练数据集，并且每次从中获取一小批（mini-batch）样本，送入模型执行前向计算得到预测值，并计算预测值（predict_label）与真实值（true_label）之间的损失函数值（loss）。执行梯度反向传播，并根据设置的优化算法（optimizer）更新模型的参数。观察每轮迭代的 loss 值减小趋势，可判断模型训练效果。

1.5.1 高层 api 实现

先用 `paddle.Model` 对模型进行封装，然后通过 `Model.fit`、`Model.evaluate`、`Model.predict` 等完成模型的训练、评估与推理。

```
[4]: # 引入 matplotlib 来可视化图片
import matplotlib as plt
import paddle
```

```
D:\anaconda\envs\paddle_env\Lib\site-
packages\paddle\utils\cpp_extension\extension_utils.py:711: UserWarning: No
ccache found. Please be aware that recompiling all source files may be required.
You can download and install ccache from:
https://github.com/ccache/ccache/blob/master/doc/INSTALL.md
warnings.warn(warning_message)
```

1. 指定训练硬件

`paddle.device.set_device` 修改。如果本机有多个 GPU 卡，也可以通过该 API 选择指定的卡进行训练，不指定的情况下则默认使用 ‘gpu:0’。

```
[20]: paddle.device.set_device("gpu:0")
```

```
[20]: Place(gpu:0)
```

只能使用 `CUDA_VISIBLE_DEVICES` 设置范围内的显卡，例如可以设置 `export CUDA_VISIBLE_DEVICES=0,1,2` 和 `paddle.device.set_device('gpu:0')`，但是设置 `export CUDA_VISIBLE_DEVICES=1` 和 `paddle.device.set_device('gpu:0')` 时会冲突报错。

2. paddle.Model 封装模型

```
[11]: # 封装模型为一个 model 实例, 便于进行后续的训练、评估和推理
paddle.disable_static() # 关闭静态图模式
model = paddle.Model(mnist)
```

3. Model.prepare 配置训练参数

- 损失函数 > 飞桨框架在 `paddle.nn Loss` 层提供了适用不同深度学习任务的损失函数相关 API。
- 优化器 > 飞桨框架在 `paddle.optimizer` 下提供了优化器相关 API。并且需要为优化器设置合适的学习率, 或者指定合适的学习率策略, 飞桨框架在 `paddle.optimizer.lr` 下提供了学习率策略相关的 API。
- 评价指标 > 飞桨框架在 `paddle.metric` 下提供了评价指标相关 API。

```
[12]: model.prepare(optimizer=paddle.optimizer.Adam(learning_rate=0.
    ↪001, parameters=model.parameters()),
        loss=paddle.nn.CrossEntropyLoss(),
        metrics=paddle.metric.Accuracy(),)
```

4.. 训练模型

做好模型训练的前期准备工作后, 调用 `Model.fit` 接口来启动训练。训练过程采用二层循环嵌套方式: 内层循环完成整个数据集的一次遍历, 采用分批次方式; 外层循环根据设置的训练轮次完成数据集的多次遍历。

因此需要指定至少三个关键参数:

训练数据集: 传入之前定义好的训练数据集。

训练轮次: 训练时遍历数据集的次数, 即外循环轮次。

每批次大小: 内循环中每个批次的训练样本数。

```
[13]: model.fit(train_dataset, epochs=5, batch_size=64, verbose=1)
```

The loss value printed in the log is the current step, and the metric is the average value of previous steps.

Epoch 1/5

D:\anaconda\envs\paddle_env\Lib\site-packages\paddle\hapi\model.py:1246:

UserWarning: To copy construct from a tensor, it is recommended to use

```

sourceTensor.clone().detach(), rather than paddle.to_tensor(sourceTensor).
    labels = [paddle.to_tensor(l) for l in to_list(labels)]

step 938/938 [=====] - loss: 0.4497 - acc: 0.8286 -
15ms/step
Epoch 2/5
step 938/938 [=====] - loss: 0.2328 - acc: 0.8899 -
11ms/step
Epoch 3/5
step 938/938 [=====] - loss: 0.1719 - acc: 0.8986 -
11ms/step
Epoch 4/5
step 938/938 [=====] - loss: 0.1708 - acc: 0.9019 -
11ms/step
Epoch 5/5
step 938/938 [=====] - loss: 0.3745 - acc: 0.9051 -
11ms/step

```

还可以设置样本乱序 (shuffle)

丢弃不完整的批次样本 (drop_last)

同步/异步读取数据 (num_workers) 等参数

另外可通过 Callback 参数传入回调函数，在模型训练的各个阶段进行一些自定义操作，比如收集训练过程中的一些数据和参数，详细介绍可参见自定义 Callback 章节。

Model.evaluate 评估模型

训练好模型后，可在事先定义好的测试数据集上，使用 Model.evaluate 接口完成模型评估操作，结束后根据在 Model.prepare 中定义的 loss 和 metric 计算并返回相关评估结果。

```

[60]: # 用 evaluate 在测试集上对模型进行验证
eval_result = model.evaluate(test_dataset, verbose=1)
print(eval_result)

```

Eval begin...

```

step 10000/10000 [=====] - loss: 7.3788e-05 - acc:
0.9183 - 3ms/step

```

Eval samples: 10000

```
{'loss': [7.378782902378589e-05], 'acc': np.float64(0.9183)}
```

返回格式是一个字典: > {'loss': xxx, 'metric name1': xxx, 'metric name2': xxx}

5. 推理 高层 API 中提供了 Model.predict 接口, 可对训练好的模型进行推理验证。只需传入待执行推理验证的样本数据, 即可计算并返回推理结果。

高层 API 中提供了 Model.predict 接口, 可对训练好的模型进行推理验证。只需传入待执行推理验证的样本数据, 即可计算并返回推理结果。

返回格式 > 如果模型是单一输出, 则输出的形状为 [1, n], n 表示数据集的样本数。其中每个 numpy_ndarray_n 是对应原始数据经过模型计算后得到的预测结果, 类型为 numpy 数组 > > 如果模型是多输出, 则输出的形状为 [m, n], m 表示标签的种类数, 在多标签分类任务中, m 会根据标签的数目而定。

```
[63]: # 用 predict 在测试集上对模型进行推理
test_result = model.predict(test_dataset)
# 由于模型是单一输出, test_result 的形状为 [1, 10000], 10000 是测试数据集的数据量。
# 这里打印第一个数据的结果, 这个数组表示每个数字的预测概率
print(len(test_result))
print(test_result[0][0])

# 从测试集中取出一张图片
img, label = test_dataset[0]

# 打印推理结果, 这里的 argmax 函数用于取出预测值中概率最高的一个的下标, 作为预测标签
pred_label = test_result[0][0].argmax()
print("true label: {}, pred label: {}".format(label[0], pred_label))
# 使用 matplotlib 库, 可视化图片
from matplotlib import pyplot as plt

plt.imshow(img[0])
```

Predict begin...

step 10000/10000 [=====] - 2ms/step

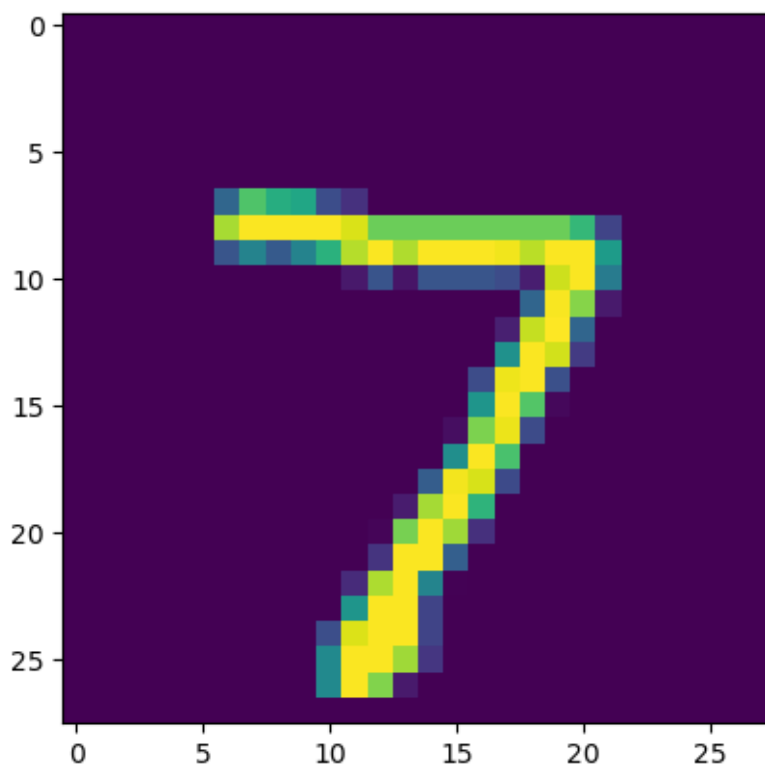
Predict samples: 10000

1

```
[[ -3.7993996 -14.965717  -3.7809212  2.3690028 -5.8950806 -1.4700184
```

```
-13.8658085  7.660885  -2.4857903  1.4592627]]  
true label: 7, pred label: 7
```

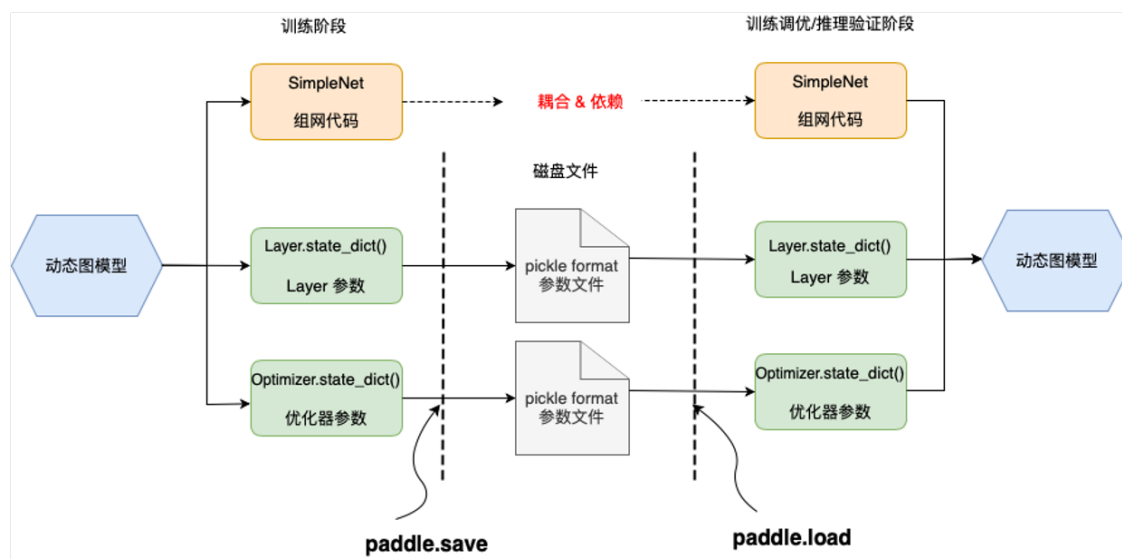
```
[63]: <matplotlib.image.AxesImage at 0x23bc006a4e0>
```



1.5.2 基础 api 实现

飞桨框架也同样支持通过基础 API。简单来说，`Model.prepare`、`Model.fit`、`Model.evaluate`、`Model.predict` 都是由基础 API 封装而来。> 具体实践中可以基础和高层组合使用，但基础 api 要麻烦些，就先不学习，以后需要再看下。

1.6 模型保存与加载



模型训练后，训练好的模型参数保存在内存中，通常需要使用模型保存（save）功能将其持久化保存到磁盘文件中，并在后续需要训练调优或推理部署时，再加载（load）到内存中运行。

API >paddle.save >>paddle.load >>paddle.jit.save >>paddle.jit.load >>paddle.Model.save >>paddle.Model.load

state_dict() 中存放了模型参数信息，包括所有可学习的和不可学习的参数（parameters 和 buffers），从网络层（Layer）和优化器（Optimizer）中获取，以字典形式存储，key 为参数名，value 为对应参数数据（Tensor）。

paddle.save: 使用 paddle.save 保存模型，实际是通过 Python pickle 模块来实现的，传入要保存的数据对象后，会在指定路径下生成一个 pickle 格式的磁盘文件。

paddle.load: 加载时还需要之前的模型组网代码，并使用 paddle.load 传入保存的文件路径，即可重新将之前保存的数据从磁盘文件中载入。

可保存的内容：> 网络层参数：Layer.state_dict() >> 优化器参数：Optimizer.state_dict() >>Tensor 数据：（如创建的 Tensor 数据、网络层的 weight 数据等）>> 含 Tensor 的 list/dict 嵌套结构对象（如保存 state_dict() 的嵌套结构对象：obj = {'model': layer.state_dict(), 'opt': adam.state_dict(), 'epoch': 100}）

1.6.1 使用高层 API

保存

[15]: # 方式一

```
model.fit(train_dataset, epochs=5, batch_size=64, verbose=1, save_freq=1)
```

The loss value printed in the log is the current step, and the metric is the average value of previous steps.

Epoch 1/5

```
step 938/938 [=====] - loss: 0.3284 - acc: 0.9059 -  
11ms/step
```

Epoch 2/5

```
step 938/938 [=====] - loss: 0.3069 - acc: 0.9059 -  
11ms/step
```

Epoch 3/5

```
step 938/938 [=====] - loss: 0.0945 - acc: 0.9058 -  
11ms/step
```

Epoch 4/5

```
step 938/938 [=====] - loss: 0.1710 - acc: 0.9085 -  
11ms/step
```

Epoch 5/5

```
step 938/938 [=====] - loss: 0.4737 - acc: 0.9077 -  
11ms/step
```

`paddle.Model.fit` 函数可自动保存模型，通过它的参数 `save_freq` 可以设置保存动态图模型的频率，即多少个 epoch 保存一次模型，默认值是 1。

[16]: # 方式二：设置训练后保存模型

```
model.save("checkpoint/test")
```

`paddle.Model.save` API。只需要传入保存的模型文件的前缀，格式如 `dirname/file_prefix` 或者 `file_prefix`，即可保存训练后的模型参数和优化器参数，保存后的文件后缀名固定为 `.pdparams` 和 `.pdopt`。

加载 高层 API 加载动态图模型所需要调用的 API 是 `paddle.Model.load`，从指定的文件中载入模型参数和优化器参数（可选）以继续训练。`paddle.Model.load` 需要传入的核心参数是待加载的模型参数或者优化器参数文件（可选）的前缀（需要保证后缀符合 `.pdparams` 和 `.pdopt`）。

[17]: `model.load("checkpoint/test")`

[21]: `!pip install visuale`

Collecting visualdl

Downloading visualdl-2.5.3-py3-none-any.whl.metadata (25 kB)

Collecting bce-python-sdk (from visualdl)

Downloading bce_python_sdk-0.9.29-py3-none-any.whl.metadata (416 bytes)

Requirement already satisfied: flask>=1.1.1 in d:\python\lib\site-packages (from visualdl) (3.1.0)

Collecting Flask-Babel>=3.0.0 (from visualdl)

Downloading flask_babel-4.0.0-py3-none-any.whl.metadata (1.9 kB)

Requirement already satisfied: numpy in d:\python\lib\site-packages (from visualdl) (2.0.2)

Requirement already satisfied: Pillow>=7.0.0 in d:\python\lib\site-packages (from visualdl) (11.0.0)

Requirement already satisfied: protobuf>=3.20.0 in d:\python\lib\site-packages (from visualdl) (5.29.1)

Requirement already satisfied: requests in d:\python\lib\site-packages (from visualdl) (2.31.0)

Requirement already satisfied: six>=1.14.0 in d:\python\lib\site-packages (from visualdl) (1.16.0)

Requirement already satisfied: matplotlib in d:\python\lib\site-packages (from visualdl) (3.10.0)

Requirement already satisfied: pandas in d:\python\lib\site-packages (from visualdl) (2.2.3)

Requirement already satisfied: packaging in d:\python\lib\site-packages (from visualdl) (24.2)

Collecting rarfile (from visualdl)

Downloading rarfile-4.2-py3-none-any.whl.metadata (4.4 kB)

Requirement already satisfied: psutil in d:\python\lib\site-packages (from visualdl) (6.1.0)

Requirement already satisfied: Werkzeug>=3.1 in d:\python\lib\site-packages (from flask>=1.1.1->visualdl) (3.1.3)

Requirement already satisfied: Jinja2>=3.1.2 in d:\python\lib\site-packages (from flask>=1.1.1->visualdl) (3.1.4)

Requirement already satisfied: itsdangerous>=2.2 in d:\python\lib\site-packages (from flask>=1.1.1->visualdl) (2.2.0)

Requirement already satisfied: click>=8.1.3 in d:\python\lib\site-packages (from flask>=1.1.1->visualdl) (8.1.7)

Requirement already satisfied: blinker>=1.9 in d:\python\lib\site-packages (from

```
flask>=1.1.1->visualdl) (1.9.0)
Requirement already satisfied: Babel>=2.12 in d:\python\lib\site-packages (from
Flask-Babel>=3.0.0->visualdl) (2.17.0)
Requirement already satisfied: pytz>=2022.7 in d:\python\lib\site-packages (from
Flask-Babel>=3.0.0->visualdl) (2024.2)
Requirement already satisfied: pycryptodome>=3.8.0 in d:\python\lib\site-
packages (from bce-python-sdk->visualdl) (3.21.0)
Collecting future>=0.6.0 (from bce-python-sdk->visualdl)
  Downloading future-1.0.0-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: contourpy>=1.0.1 in d:\python\lib\site-packages
(from matplotlib->visualdl) (1.3.1)
Requirement already satisfied: cycycler>=0.10 in d:\python\lib\site-packages (from
matplotlib->visualdl) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in d:\python\lib\site-packages
(from matplotlib->visualdl) (4.55.1)
Requirement already satisfied: kiwisolver>=1.3.1 in d:\python\lib\site-packages
(from matplotlib->visualdl) (1.4.7)
Requirement already satisfied: pyparsing>=2.3.1 in d:\python\lib\site-packages
(from matplotlib->visualdl) (3.1.4)
Requirement already satisfied: python-dateutil>=2.7 in d:\python\lib\site-
packages (from matplotlib->visualdl) (2.9.0.post0)
Requirement already satisfied: tzdata>=2022.7 in d:\python\lib\site-packages
(from pandas->visualdl) (2024.2)
Requirement already satisfied: charset-normalizer<4,>=2 in d:\python\lib\site-
packages (from requests->visualdl) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in d:\python\lib\site-packages (from
requests->visualdl) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in d:\python\lib\site-packages
(from requests->visualdl) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in d:\python\lib\site-packages
(from requests->visualdl) (2024.8.30)
Requirement already satisfied: colorama in d:\python\lib\site-packages (from
click>=8.1.3->flask>=1.1.1->visualdl) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in d:\python\lib\site-packages
(from Jinja2>=3.1.2->flask>=1.1.1->visualdl) (3.0.2)
Downloading visualdl-2.5.3-py3-none-any.whl (6.3 MB)
----- 0.0/6.3 MB ? eta -:-:--
```

```

----- 0.0/6.3 MB ? eta -:--:--
----- 0.0/6.3 MB ? eta -:--:--
--- ----- 0.5/6.3 MB 2.8 MB/s eta 0:00:03
----- ----- 1.0/6.3 MB 2.4 MB/s eta 0:00:03
----- ----- 1.6/6.3 MB 2.4 MB/s eta 0:00:02
----- ----- 2.1/6.3 MB 2.3 MB/s eta 0:00:02
----- ----- 2.4/6.3 MB 2.2 MB/s eta 0:00:02
----- ----- 2.6/6.3 MB 2.0 MB/s eta 0:00:02
----- ----- 2.9/6.3 MB 2.0 MB/s eta 0:00:02
----- ----- 3.1/6.3 MB 1.9 MB/s eta 0:00:02
----- ----- 3.4/6.3 MB 1.9 MB/s eta 0:00:02
----- ----- 3.7/6.3 MB 1.8 MB/s eta 0:00:02
----- ----- 3.9/6.3 MB 1.7 MB/s eta 0:00:02
----- ----- 4.2/6.3 MB 1.7 MB/s eta 0:00:02
----- ----- 4.5/6.3 MB 1.6 MB/s eta 0:00:02
----- ----- 4.7/6.3 MB 1.6 MB/s eta 0:00:02
----- ----- 4.7/6.3 MB 1.6 MB/s eta 0:00:02
----- ----- 5.0/6.3 MB 1.5 MB/s eta 0:00:01
----- ----- 5.0/6.3 MB 1.5 MB/s eta 0:00:01
----- ----- 5.2/6.3 MB 1.4 MB/s eta 0:00:01
----- ----- 5.2/6.3 MB 1.4 MB/s eta 0:00:01
----- ----- 5.5/6.3 MB 1.3 MB/s eta 0:00:01
----- ----- 5.5/6.3 MB 1.3 MB/s eta 0:00:01
----- ----- 5.8/6.3 MB 1.2 MB/s eta 0:00:01
----- ----- 5.8/6.3 MB 1.2 MB/s eta 0:00:01
----- ----- 5.8/6.3 MB 1.2 MB/s eta 0:00:01
----- ----- 6.0/6.3 MB 1.1 MB/s eta 0:00:01
----- ----- 6.0/6.3 MB 1.1 MB/s eta 0:00:01
----- ----- 6.0/6.3 MB 1.1 MB/s eta 0:00:01
----- ----- 6.3/6.3 MB 1.0 MB/s eta 0:00:01
----- ----- 6.3/6.3 MB 1.0 MB/s eta 0:00:00

Downloading flask_babel-4.0.0-py3-none-any.whl (9.6 kB)
Downloading bce_python_sdk-0.9.29-py3-none-any.whl (343 kB)
Downloading rarfile-4.2-py3-none-any.whl (29 kB)
Downloading future-1.0.0-py3-none-any.whl (491 kB)
Installing collected packages: rarfile, future, bce-python-sdk, Flask-Babel,
visuall

```

Successfully installed Flask-Babel-4.0.0 bce-python-sdk-0.9.29 future-1.0.0
rarfile-4.2 visuddl-2.5.3

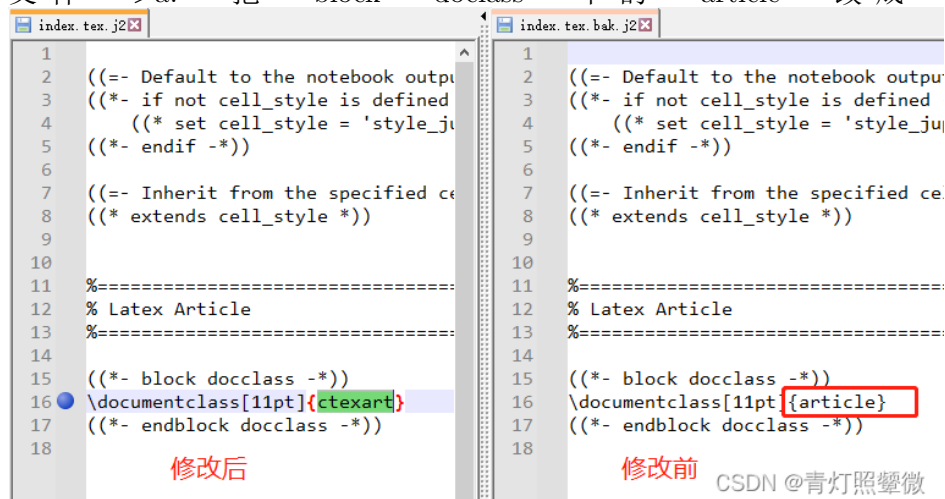
2 番外相关工作

2.1 jupyter notebook 导出 pdf 中文消失问题

在导出本总结时，遇到了 jupyter notebook 中文无法正常导出的问题，参考[文章](#)，但并未完全按照它的，以下是我的改进方案：

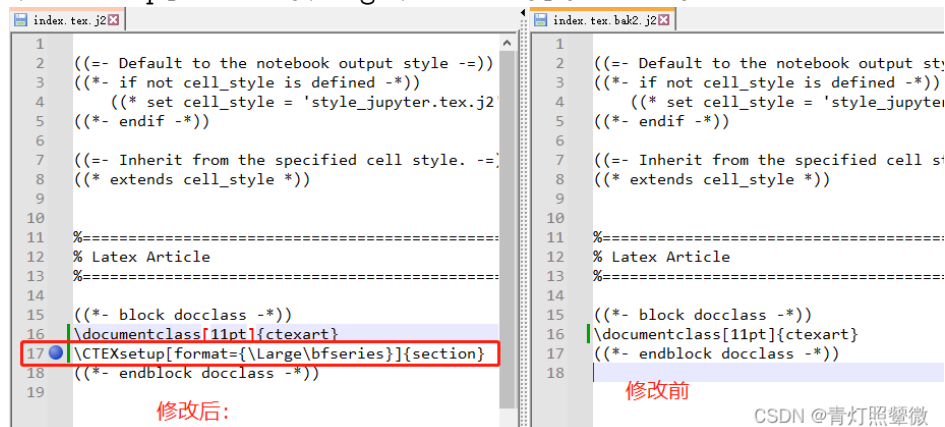
1. 安装
2. 安装miktex
3. 找到 Python\share\jupyter\nbconvert\templates\latex\index.tex.j2

文件 >a. 把 block docclass 下的 article 改成 ctexart



>b. 在 index.tex.j2 文件中增加如下命令：

\CTEXsetup[format={\Large\bfseries}]{section}



3 下周工作

1. 继续完成 pp 的实践练习。
2. 配置学校算力平台的环境，初步学习 docker，命令行 ssh 等。
3. 借助算力平台初步还原跑通 Times former